Void Companion: Binary Distinction and the Four-Vertex Closure

Johannes Michael Wielsch
Companion note to *Void and Form*
Project DOI: 10.5281/zenodo.19491035

May 2026

Abstract

This note is the compact map and self-contained formal kernel of *Void and Form*, proposed as a theory of formulability: an account of the conditions under which mathematical, logical, and physical description can be stated without collapse. The note does not ask what emerges from a presupposed "nothing". It asks under which conditions formal description is possible at all. The pre-formal answer is the availability of distinguishable positions: without them no relation, judgement, equation, map, representation, truth-value, or falsehood can be formulated. *Void* names this boundary. It is not the empty set, not the empty type, and not a physical nothing; it is the absence of the domain in which those notions could already occur.

The checked kernel begins once distinguishability is granted as data. In intensional Martin-Löf type theory, a binary distinction is represented by two non-collapsing points that exhaust their carrier. This carrier has canonical normal form Two; its endomorphisms exhaust to four cases; representing those cases with separation, complete realisation, and no surplus yields a four-vertex carrier with complete simple adjacency; and every such closure points back to the data from which it was built. In addition, the cardinality of the entry datum is itself made formal: a singleton carrier admits no Distinction; the original record is exactly the $n = 2$ case of an n-ary template; and at $n = 3$ the n-ary record is canonically recoverable from a binary package of three pairwise separations and a nested binary cover, with the displayed boundary fields closing by checked equality.

The formulability claim is therefore not treated as a theorem of this note. It is the architectural reading of the checked chain. The note itself records the conditional formal core: if the formal threshold is crossed by supplying the data of binary distinction, then the displayed closure and dependency results follow. The thesis that distinction is foundational is at least as old as Spencer-Brown's *Laws of Form* and is not asserted here as new. The contribution is the precise, machine-checked statement that, under the explicitly fixed conditions of representation, the closure of binary distinction is K_4, unique up to isomorphism, and dependent back on the data from which it was built.

How to read this note

This is the self-contained formal kernel of *Void and Form*, not the full development. It can be read before the larger source. For readability, it uses standard-library imports for equality, sums, products, the empty type, and negation. The full file `Void.lagda.tex` rebuilds that infrastructure internally and continues with constructions that are out of scope for this note.

The note has three layers. First, it names the claim that motivates the project: *Void and Form* is a proposed theory of formulability, not a proof that physics has already been derived. Second, it names the pre-formal boundary at which formalisation becomes possible at all: relations, morphisms, judgements, and equations already presuppose a field in which one place can be separated

from another. This is the conceptual reason for taking binary distinction as the starting datum; it is not an Agda theorem. Third, it records the checked statements: normal form, exhaustion, four-vertex closure, and dependency back to the starting data.

The code is narrower than the larger claim, and that is the point. It starts only after the data are granted. From that point on, every formal claim below is a term checked by Agda. The tetrahedron is only the geometric reading of the complete graph on the four endomorphism cases; it is not an additional assumption used in the proof.

Map of the project

This note is also the table of contents for the project in compressed form. The map below is not a second proof. It fixes the status of the layers so that the reader can see where *Void* ends, where the formal threshold begins, and where *Form* later becomes interpretation.

Void: conditions of formulability

Level 0: Void. No type, no term, no carrier. The schematic marker is $\mathcal{V} \notin Set_\omega$. Void is not a stable endpoint in the sense of a foundation, because every act of positing already presupposes structure. It is therefore conceptually non-closable. This level is marked, not proved.

Level 1: Information. Information is understood in Bateson's minimal sense: a difference that makes a difference. Without distinction, an occurrence carries no information. Information therefore presupposes distinction, and an inhabited distinction requires at least two distinguishable occurrences. This binary shape is minimal because every further difference already relies on it.

Void: formal execution

Level 2: Two. The first stable formal carrier is Two. It contains two separated points and no third generator. It is not a truth-value object; it is the normal form of the Distinction record. From this level onward the formal threshold is crossed: MLTT and Agda execute the consequences of the datum they have received.

Level 3: Logic. Formal systems operate on structures in which distinguishable positions are already available. Relation, morphism, judgement, and equation do not construct that availability; they use it. MLTT is the clean carrier used here, not the origin of the cut.

Level 4: Mathematics. The machine-checked chain is the mathematical core: every Distinction is isomorphic to Two; the endomorphisms of Two have exactly four cases; representing those cases under separation, complete realisation, and no surplus forces the four-element closure, unique up to isomorphism; and the closure returns the originating distinction. No further generator is introduced.

Form: structural reading and interpretation

Level 5: Structural separation. Two distinguishable occurrences collapse if neither origin nor position can distinguish them. Non-identity of origin gives the germ of temporal order; non-coincidence gives the germ of spatial separation. Space and time are not asserted as primitive entities here. They name conditions under which distinction does not collapse.

Level 6: Physics. Physics enters as interpretation of the preceding structural conditions by an observer who already operates inside them. Form is not physics; it is the structure from which physical description can be read.

Level 7: Contingency. Necessity stops before complete worldly specification. The preceding levels fix conditions, not the full concrete world that satisfies them.

Level 8: Induction. No new external generator is postulated. More complex structures are read as repeated applications and stabilisations of the original distinction. Induction is the law of that repetition. This is a structural reading, not a reductionist claim about all mathematics.

Levels 0 and 1 are not formal statements inside a type system. They state the conditions under which formal statements can appear. From Level 2 onward the formal theory begins, and the claims are expressible and checkable inside it.

Part I. The pre-formal threshold

The pre-formal boundary

The project begins before the point at which a formal theory can already speak in its own voice. A relation has two positions. A morphism has a source and a target. A judgement has a term and a type. An equation has two sides, even when it proves them equal. Every such form is already a use of separation. A theory may analyse that separation once it has been given; it cannot produce the very field of separability from within a field that it already presupposes.

The following display is therefore not Agda code and not a theorem of MLTT. It is a schematic boundary marker: it states where the proposed theory of formulability reaches its formal threshold. The names in the display are not internal definitions, and the arrows are not checked implication terms; they are meta-level dependency markers for the passage into the formal part.

$$
\begin{array}{ll}
\text{Level 0 :} & \mathcal{V} \notin \mathit{Set}_\omega, \qquad \neg\exists x\,(x : \mathcal{V}). \\
\text{Level 1 :} & \mathrm{Sing}(S) := \exists p : S\,\forall x : S\,(x = p). \\
\text{Collapse :} & \mathrm{Sing}(S) \Longrightarrow \neg\exists a, b : S\,(a \neq b). \\
\text{Level 2 :} & \mathrm{Dist}(S) := \exists a, b : S\,(a \neq b \ \wedge\ \forall x : S\,(x = a \vee x = b)). \\
\text{Separation :} & a \neq b \Longrightarrow \neg\mathrm{SameOrigin}(a, b) \ \wedge\ \neg\mathrm{Contact}(a, b). \\
\text{Formal threshold :} & \mathrm{Dist}(S) \Longrightarrow \begin{array}{l} S \simeq \mathbf{2}, \\ \mathrm{End}(\mathbf{2}) = \{c_L, c_R, \mathrm{id}, \sigma\}, \\ \text{representation closure} = K_4, \\ K_4 \Longrightarrow \mathrm{Dist}(S). \end{array}
\end{array}
$$

Level 0 is not a hidden type. The symbol $\mathcal{V}$ marks the absence of a domain in which a term could already be typed. Level 1 shows why mere inhabitance is still collapse: if all occurrences are the same occurrence, no difference can be carried internally. Level 2 is the first non-collapsing datum: two separated occurrences and no third generator. Here, $\mathrm{SameOrigin}(a, b)$ names the collapse in which the two occurrences have no distinguishable order of arising; $\mathrm{Contact}(a, b)$ names the collapse in which no gap separates their positions. The two negations in the separation line

mark the ontological content that the later volume *Form* reads as the germ of temporal and spatial structure: without an interval of origin or a gap of contact, the two occurrences collapse back into one.

This boundary is not a theorem of the development; it is the condition under which the development can be read as a theorem at all. This is also why the formulability claim is architectural rather than merely decorative: the claim names the whole dependency from formulability to formal closure, while the code checks the segment that can be expressed as terms. Only the last line enters Agda. From Dist(S) onward, the schematic notation is replaced by checked terms. The rest of this note records that checked consequence.

Why two cannot occur without separation

Before any cardinality is even chosen, a sharper question arises. Why can two occurrences not already be present at the threshold without any separating relation between them? In the absence of time, space, or any other field of differentiation, why does the supposed pair not simply coexist as two?

The answer is not specific to type theory and not specific to any single foundation. It is the principle that what cannot be told apart is not two. Leibniz stated it in 1686 as the *principium identitatis indiscernibilium*: indiscernibles are identical. The same content reappears, in the technical idiom of each framework, wherever "two" is given a precise sense:

- In classical first-order logic with equality, substitutivity of identity makes any two elements that satisfy the same predicates equal.
- In set theory, the axiom of extensionality says that two sets with the same elements are the same set.
- In category theory, the Yoneda principle expresses that an object is determined by its relations to others; an object with no distinguishing relation is not a second object.
- In quantum mechanics, particles in the same state are not two entities that happen to coincide; the statistical structure of the theory (Bose, Fermi) is forced precisely because indistinguishables cannot be counted as two.

Each of these frameworks pays the same price: as soon as "two distinct X" is even said, separation has already been used. No foundation has exhibited a coherent counterexample, because exhibition itself requires a distinguishing predicate.

The consequence is immediate. Whatever has no separating predicate is, by the principle that defines what equality of occurrences means, one occurrence and not two. The first non-collapsing pair and the first separating predicate arrive together; there is no temporal, spatial, or relational gap in which two might quietly coexist before separation. The absence of any separating predicate forces collapse to one.

The companion does not assume separation as a primitive act in the manner of *Laws of Form*; it relies on a principle that any framework supporting the very statement "two distinct X" already concedes. How this principle becomes a checkable term once a specific carrier is chosen is shown later, in the section on the executed Leibniz collapse, after the formal infrastructure of this note is in place.

Why binary, not unary or ternary

The choice of binary distinction as the entry datum is not stipulation and not preference. At the pre-formal threshold, only one cardinality survives the joint demand of non-collapse and self-sufficiency. This argument is conceptual, not an Agda theorem; the formal results below do not depend on it. It is recorded here so that the antecedent of the machine-checked conditional is not itself opaque.

Unary collapses. A single occurrence carries no separation. The schematic predicate $\mathrm{Sing}(S)$ records exactly this: every element equals the chosen point, and no pair (a, b) with $a \neq b$ can be exhibited. Whatever appears within S is the same appearance. Formalisation cannot start here, because relations, morphisms, judgements, and equations all require two positions before they can be written down.

Ternary carries derived binary plus surplus. Three positions a, b, c require three pairwise separations $a \neq b$, $b \neq c$, $a \neq c$. Each pairwise separation is itself a binary distinction. A primitive ternary datum would have to supply, on top of these three binary facts, an additional structure (an order, an incidence, a selection of one position against the other two) that the act of distinguishing alone does not yield. Any device that inspects three positions simultaneously is itself a fourth position relative to the three, and its application to them again splits as "this one against the other two". The ternary case therefore decomposes into binary distinctions plus surplus structure that requires its own justification.

Binary survives. A binary distinction supplies exactly one separation $a \neq b$ together with the exhaustion clause that nothing further is present. No additional structure (order, incidence, selection) is assumed; any such structure that arises later is forced by the four endomorphism cases of the normal form, not added to the datum. This is why the formal development below begins with Distinction and not with a richer or poorer record.

These three observations are made formal in Part II, in the section on the cardinality of distinction: a singleton carrier admits no inhabitant of Distinction, the n-ary version of the record is at $n = 2$ mutually convertible with Distinction, and higher arities carry only pairwise binary separation witnesses together with a nested binary disjunction.

Part II. The formal threshold

1 Setting

The setting is ordinary intensional Martin-Löf type theory as implemented in Agda. The code snippets below use the standard library for equality, sums, products, the empty type, and negation. No attempt is made here to reproduce the full formal development; the goal is to record the minimal spine of the argument. The carrier of the binary distinction is introduced as a fresh type Two with constructors L and R, rather than reusing Bool. The two types have the same shape, but Bool carries an established reading as truth-values. The result below depends on separation alone, not on any prior reading of the two points as truth or falsity, and the renaming makes this independence explicit.

```
{-# OPTIONS --safe --without-K #-}

module VoidCompanion where

open import Relation.Binary.PropositionalEquality using (_≡_; refl; sym; trans; cong)
open import Data.Empty using (⊥; ⊥-elim)
```

```
open import Data.Sum using (_⊎_; inj₁; inj₂)
open import Data.Product using (Σ; _,_)
open import Relation.Nullary using (¬_)
```

The two-point normal form is the following type.

```
data Two : Set where
 L R : Two

L≠R : ¬ (L ≡ R)
L≠R ()

R≠L : ¬ (R ≡ L)
R≠L p = L≠R (sym p)
```

2 Distinction

A binary distinction is not just a type with two named points. The points must be separated, and every element of the carrier must be one of them. This is the exact hypothesis that makes the normal-form theorem true.

```
record Distinction : Set₁ where
 field
  S         : Set
  left      : S
  right     : S
  separated : ¬ (left ≡ right)
  cover     : (x : S) → (x ≡ left) ⊎ (x ≡ right)
```

The canonical inhabitant is obtained from Two itself.

```
Two-cover : (x : Two) → (x ≡ L) ⊎ (x ≡ R)
Two-cover L = inj₁ refl
Two-cover R = inj₂ refl

Two-distinction : Distinction
Two-distinction = record
 { S         = Two
 ; left      = L
 ; right     = R
 ; separated = L≠R
 ; cover     = Two-cover
 }
```

3 Cardinality of distinction

The earlier prose in Part I argued that one occurrence collapses, three occurrences decompose, and two survives. The argument was explicitly marked as conceptual. At this point, after the Distinction record is in place, the same content can be stated and proved formally. Two facts together turn the binary choice from preference into a theorem of the present setting: a singleton carrier admits no inhabitant of Distinction at all, and the n-ary generalisation of the record is, at $n = 2$, mutually convertible with the binary case. Higher arities carry only the pairwise binary separation witnesses one already has, plus an exhaustive cover that is structurally a nested binary disjunction. No primitive ternary or higher-arity operator is required.

```
open import Data.Nat using (ℕ; zero; suc)
open import Data.Fin using (Fin) renaming (zero to fz; suc to fs)
```

```
record Distinction-n (n : ℕ) : Set₁ where
 field
  S       : Set
  point   : Fin n → S
  distinct : (i j : Fin n) → ¬ (i ≡ j) → ¬ (point i ≡ point j)
  cover   : (x : S) → Σ (Fin n) (λ i → x ≡ point i)
```

Unary collapses formally. No singleton carrier hosts a Distinction: if every two elements of the carrier are equal, the boundary points are equal and separated is falsified.

```
singleton-no-distinction :
 (d : Distinction) →
 ((x y : Distinction.S d) → x ≡ y) →
 ⊥
singleton-no-distinction d hyp =
 Distinction.separated d (hyp (Distinction.left d) (Distinction.right d))
```

The same content holds for Distinction-n at $n = 1$: any two elements of its carrier are forced equal, so no separation between distinct positions can be inhabited.

```
distinction-n-1-collapses :
 (d : Distinction-n 1) →
 (x y : Distinction-n.S d) → x ≡ y
distinction-n-1-collapses d x y
 with Distinction-n.cover d x | Distinction-n.cover d y
... | fz , px | fz , py = trans px (sym py)
... | fz , _ | fs () , _
... | fs () , _ | _
```

The collapse on Distinction-n 1 is one half of the "unary kills distinction" content; on its own, it forces every two elements of the carrier to coincide. The bridge to the original record is the next step: any Distinction that injects into such a carrier is immediately impossible, because the two separated boundary points are forced equal under the injection and separated is contradicted. This makes the joint statement, that no Distinction survives on a unary carrier, a single checked term and not a verbal combination of two lemmas.

```
no-distinction-on-Dn1 :
 (d1 : Distinction-n 1) →
 (d : Distinction) →
 (ϕ : Distinction.S d → Distinction-n.S d1) →
 ((x y : Distinction.S d) → ϕ x ≡ ϕ y → x ≡ y) →
 ⊥
no-distinction-on-Dn1 d1 d ϕ ϕ-inj =
 Distinction.separated d
  (ϕ-inj (Distinction.left d) (Distinction.right d)
    (distinction-n-1-collapses d1
     (ϕ (Distinction.left d))
     (ϕ (Distinction.right d))))
```

Binary equals the existing record. At $n = 2$ the n-ary record is mutually convertible with the original Distinction. The two constructions are mechanical and demonstrate that no boundary information is lost in either direction. Because the records contain function fields, the companion does not claim an unqualified record equality of the conversions; instead it states the recoverable carrier and boundary fields explicitly.

```
distinction-to-Dn2 : Distinction → Distinction-n 2
distinction-to-Dn2 d = record
 { S       = Distinction.S d
 ; point   = pt
 ; distinct = dist
 ; cover   = cov
 }
 where
```

```
  open Distinction d
  pt : Fin 2 → S
  pt fz = left
  pt (fs fz) = right

  dist : (i j : Fin 2) → ¬ (i ≡ j) → ¬ (pt i ≡ pt j)
  dist fz fz       h _ = h refl
  dist fz (fs fz) _ p = separated p
  dist (fs fz) fz _ p = separated (sym p)
  dist (fs fz) (fs fz) h _ = h refl

  cov : (x : S) → Σ (Fin 2) (λ i → x ≡ pt i)
  cov x with cover x
  ... | inj₁ p = fz , p
  ... | inj₂ p = fs fz , p

Dn2-to-distinction : Distinction-n 2 → Distinction
Dn2-to-distinction d = record
 { S       = Distinction-n.S d
 ; left    = Distinction-n.point d fz
 ; right   = Distinction-n.point d (fs fz)
 ; separated = Distinction-n.distinct d fz (fs fz) λ ()
 ; cover = cov
 }
 where
  open Distinction-n d
  cov : (x : S) → (x ≡ point fz) ⊎ (x ≡ point (fs fz))
  cov x with cover x
  ... | fz , p      = inj₁ p
  ... | fs fz , p = inj₂ p

record DistinctionDn2Equivalence : Set₁ where
 field
  from-to-carrier : (d : Distinction) →
   Distinction.S (Dn2-to-distinction (distinction-to-Dn2 d)) ≡ Distinction.S d
  from-to-left      : (d : Distinction) →
   Distinction.left (Dn2-to-distinction (distinction-to-Dn2 d)) ≡ Distinction.left d
  from-to-right     : (d : Distinction) →
   Distinction.right (Dn2-to-distinction (distinction-to-Dn2 d)) ≡ Distinction.right d
  to-from-carrier : (d : Distinction-n 2) →
   Distinction-n.S (distinction-to-Dn2 (Dn2-to-distinction d)) ≡ Distinction-n.S d
  to-from-point-0 : (d : Distinction-n 2) →
   Distinction-n.point (distinction-to-Dn2 (Dn2-to-distinction d)) fz ≡ Distinction-n.point d fz
  to-from-point-1 : (d : Distinction-n 2) →
   Distinction-n.point (distinction-to-Dn2 (Dn2-to-distinction d)) (fs fz) ≡ Distinction-n.point d (fs fz)

theorem-distinction-Dn2-equivalence : DistinctionDn2Equivalence
theorem-distinction-Dn2-equivalence = record
 { from-to-carrier = λ _ → refl
 ; from-to-left    = λ _ → refl
 ; from-to-right = λ _ → refl
 ; to-from-carrier = λ _ → refl
 ; to-from-point-0 = λ _ → refl
 ; to-from-point-1 = λ _ → refl
 }
```

Higher arity carries only pairwise binary plus a nested disjunction. At any n, the n-ary distinction record carries a binary separation witness for every ordered pair of distinct positions. This witness is the distinct field itself; no further structure is needed to extract it.

```
pairwise-binary :
 {n : ℕ} (d : Distinction-n n) →
 (i j : Fin n) → ¬ (i ≡ j) →
 ¬ (Distinction-n.point d i ≡ Distinction-n.point d j)
pairwise-binary = Distinction-n.distinct
```

At $n = 3$, the exhaustive cover of three positions is structurally the nested binary disjunction $A_0 \uplus (A_1 \uplus A_2)$, where A_i records that an element equals position i. No primitive ternary disjunction is invoked; the three-case split is two binary splits.

```
cover-3-nested :
 (d : Distinction-n 3) (x : Distinction-n.S d) →
 (x ≡ Distinction-n.point d fz)
  ∪ ((x ≡ Distinction-n.point d (fs fz))
     ∪ (x ≡ Distinction-n.point d (fs (fs fz))))
cover-3-nested d x with Distinction-n.cover d x
... | fz , p          = inj₁ p
... | fs fz , p       = inj₂ (inj₁ p)
... | fs (fs fz) , p = inj₂ (inj₂ p)
```

The two preceding lemmas show only that a ternary record *carries* binary content. The stronger claim, that the ternary record is *nothing more* than this binary content, requires the converse direction: any package of three pairwise separations together with a binary-nested cover must reconstruct a Distinction-n 3. The following record names exactly the binary data extractable from a ternary distinction, and the two conversions below show the reduction in both directions.

```
record Dn3-binary-data : Set₁ where
 field
  S    : Set
  p0  : S
  p1  : S
  p2  : S
  sep01 : ¬ (p0 ≡ p1)
  sep02 : ¬ (p0 ≡ p2)
  sep12 : ¬ (p1 ≡ p2)
  cover : (x : S) → (x ≡ p0) ∪ ((x ≡ p1) ∪ (x ≡ p2))

Dn3-to-binary : Distinction-n 3 → Dn3-binary-data
Dn3-to-binary d = record
 { S   = Distinction-n.S d
 ; p0 = Distinction-n.point d fz
 ; p1 = Distinction-n.point d (fs fz)
 ; p2 = Distinction-n.point d (fs (fs fz))
 ; sep01 = Distinction-n.distinct d fz (fs fz) λ ()
 ; sep02 = Distinction-n.distinct d fz (fs (fs fz)) λ ()
 ; sep12 = Distinction-n.distinct d (fs fz) (fs (fs fz)) λ ()
 ; cover = cover-3-nested d
 }

binary-to-Dn3 : Dn3-binary-data → Distinction-n 3
binary-to-Dn3 b = record
 { S       = S
 ; point  = pt
 ; distinct = dist
 ; cover = cov
 }
 where
  open Dn3-binary-data b
  pt : Fin 3 → S
  pt fz           = p0
  pt (fs fz)      = p1
  pt (fs (fs fz)) = p2

  dist : (i j : Fin 3) → ¬ (i ≡ j) → ¬ (pt i ≡ pt j)
  dist fz          fz          h _ = h refl
  dist fz          (fs fz)     _ p = sep01 p
  dist fz          (fs (fs fz)) _ p = sep02 p
  dist (fs fz)     fz          _ p = sep01 (sym p)
  dist (fs fz)     (fs fz)     h _ = h refl
  dist (fs fz)     (fs (fs fz)) _ p = sep12 p
  dist (fs (fs fz)) fz          _ p = sep02 (sym p)
  dist (fs (fs fz)) (fs fz)     _ p = sep12 (sym p)
  dist (fs (fs fz)) (fs (fs fz)) h _ = h refl

  cov : (x : S) → Σ (Fin 3) (λ i → x ≡ pt i)
  cov x with cover x
  ... | inj₁ p          = fz , p
  ... | inj₂ (inj₁ p)   = fs fz , p
```

```
  ... | inj₂ (inj₂ p)     = fs (fs fz) , p

record Dn3BinaryReconstruction : Set₁ where
 field
  extracted-carrier : (d : Distinction-n 3) →
   Dn3-binary-data.S (Dn3-to-binary d) ≡ Distinction-n.S d
  extracted-point-0 : (d : Distinction-n 3) →
   Dn3-binary-data.p0 (Dn3-to-binary d) ≡ Distinction-n.point d fz
  extracted-point-1 : (d : Distinction-n 3) →
   Dn3-binary-data.p1 (Dn3-to-binary d) ≡ Distinction-n.point d (fs fz)
  extracted-point-2 : (d : Distinction-n 3) →
   Dn3-binary-data.p2 (Dn3-to-binary d) ≡ Distinction-n.point d (fs (fs fz))
  reconstructed-carrier : (b : Dn3-binary-data) →
   Distinction-n.S (binary-to-Dn3 b) ≡ Dn3-binary-data.S b
  reconstructed-point-0 : (b : Dn3-binary-data) →
   Distinction-n.point (binary-to-Dn3 b) fz ≡ Dn3-binary-data.p0 b
  reconstructed-point-1 : (b : Dn3-binary-data) →
   Distinction-n.point (binary-to-Dn3 b) (fs fz) ≡ Dn3-binary-data.p1 b
  reconstructed-point-2 : (b : Dn3-binary-data) →
   Distinction-n.point (binary-to-Dn3 b) (fs (fs fz)) ≡ Dn3-binary-data.p2 b

theorem-Dn3-binary-reconstruction : Dn3BinaryReconstruction
theorem-Dn3-binary-reconstruction = record
 { extracted-carrier = λ _ → refl
 ; extracted-point-0 = λ _ → refl
 ; extracted-point-1 = λ _ → refl
 ; extracted-point-2 = λ _ → refl
 ; reconstructed-carrier = λ _ → refl
 ; reconstructed-point-0 = λ _ → refl
 ; reconstructed-point-1 = λ _ → refl
 ; reconstructed-point-2 = λ _ → refl
 }
```

What these proofs together establish is the formal counterpart of *Why binary, not unary or ternary* from Part I, in the precise sense available to the present framework. Singleton carriers cannot host a distinction at all: the lemma distinction-n-1-collapses forces every two elements of a unary carrier to coincide, and no-distinction-on-Dn1 closes the bridge by showing that no Distinction can inject into such a carrier without contradicting separated. The original record is exactly the $n = 2$ case of an n-ary template (distinction-to-Dn2, Dn2-to-distinction), with the carrier and boundary roundtrips bundled in theorem-distinction-Dn2-equivalence. At $n = 3$, the conversions Dn3-to-binary and binary-to-Dn3 prove more than that ternary data carries binary content: they prove that ternary data is recoverable from binary content, since the binary package of three pairwise separations and a nested binary cover reconstructs a Distinction-n 3. The recoverable carrier and boundary fields are collected in theorem-Dn3-binary-reconstruction.

The scope of this last claim is precisely the scope of the framework. The theorem says that, inside this development, every inhabitant of Distinction-n 3 is recoverable from a binary package at the displayed carrier and boundary fields; it does not assert that all function fields are definitionally identical after a roundtrip, nor that no other foundation could treat "three positions" as a primitive entity. An ontology that posits an irreducible ternary primitive is not refuted here. What is shown is that within the present record-based, –safe –without-K fragment of MLTT, the n-ary record at $n = 3$ adds no information beyond binary data plus a labelling. Binary is therefore the minimal cardinality at which the Distinction record is inhabitable in this framework, and higher arities reduce, in this framework, to binary structure plus a labelling map.

4 Normal Form

An isomorphism of distinctions preserves the two boundary roles and has inverse maps on carriers. The normal-form theorem says that every distinction is isomorphic, in this sense, to the canonical distinction on Two.

```
record DistinctionIso (d e : Distinction) : Set₁ where
 private
  module D = Distinction d
  module E = Distinction e
 field
  to        : D.S → E.S
  from      : E.S → D.S
  to-left   : to D.left ≡ E.left
  to-right  : to D.right ≡ E.right
  from-left : from E.left ≡ D.left
  from-right : from E.right ≡ D.right
  to-from   : (y : E.S) → to (from y) ≡ y
  from-to   : (x : D.S) → from (to x) ≡ x
```

The map to Two is forced by the cover: an element is sent to L if it is the left boundary and to R if it is the right boundary.

```
toTwo : (d : Distinction) → Distinction.S d → Two
toTwo d x with Distinction.cover d x
... | inj₁ _ = L
... | inj₂ _ = R

fromTwo : (d : Distinction) → Two → Distinction.S d
fromTwo d L = Distinction.left d
fromTwo d R = Distinction.right d

toTwo-left : (d : Distinction) → toTwo d (Distinction.left d) ≡ L
toTwo-left d with Distinction.cover d (Distinction.left d)
... | inj₁ _ = refl
... | inj₂ p = ⊥-elim (Distinction.separated d p)

toTwo-right : (d : Distinction) → toTwo d (Distinction.right d) ≡ R
toTwo-right d with Distinction.cover d (Distinction.right d)
... | inj₁ p = ⊥-elim (Distinction.separated d (sym p))
... | inj₂ _ = refl

toTwo-fromTwo : (d : Distinction) → (y : Two) → toTwo d (fromTwo d y) ≡ y
toTwo-fromTwo d L = toTwo-left d
toTwo-fromTwo d R = toTwo-right d

fromTwo-toTwo : (d : Distinction) → (x : Distinction.S d) → fromTwo d (toTwo d x) ≡ x
fromTwo-toTwo d x with Distinction.cover d x
... | inj₁ p = sym p
... | inj₂ p = sym p

two-normal-form : (d : Distinction) → DistinctionIso d Two-distinction
two-normal-form d = record
 { to        = toTwo d
 ; from      = fromTwo d
 ; to-left   = toTwo-left d
 ; to-right  = toTwo-right d
 ; from-left = refl
 ; from-right = refl
 ; to-from   = toTwo-fromTwo d
 ; from-to   = fromTwo-toTwo d
 }

record NormalFormEqualityDiscipline (d : Distinction) : Set where
 field
  fromTwo-L-definitional : fromTwo d L ≡ Distinction.left d
  fromTwo-R-definitional : fromTwo d R ≡ Distinction.right d
  toTwo-left-propositional : toTwo d (Distinction.left d) ≡ L
  toTwo-right-propositional : toTwo d (Distinction.right d) ≡ R
  toTwo-fromTwo-propositional : (y : Two) → toTwo d (fromTwo d y) ≡ y
  fromTwo-toTwo-propositional : (x : Distinction.S d) → fromTwo d (toTwo d x) ≡ x

theorem-normal-form-equality-discipline :
 (d : Distinction) → NormalFormEqualityDiscipline d
theorem-normal-form-equality-discipline d = record
```

```
{ fromTwo-L-definitional = refl
; fromTwo-R-definitional = refl
; toTwo-left-propositional = toTwo-left d
; toTwo-right-propositional = toTwo-right d
; toTwo-fromTwo-propositional = toTwo-fromTwo d
; fromTwo-toTwo-propositional = fromTwo-toTwo d
}
```

This is the whole normal-form argument. No choice is made in the map: the cover field gives only two possible cases, and the separation field rules out the contradictory boundary cases. The equality discipline is explicit. The map fromTwo computes by pattern matching, so its two endpoint equations close by refl. The equations involving toTwo are propositional equalities: they use the supplied cover and, when the wrong boundary is selected, the separation witness eliminates the impossible branch.

5 Endomorphisms of Two

An endomorphism of Two is fixed by its two values. There are therefore four cases: constant left, constant right, identity, and swap.

```
data EndoCase : Set where
 constL : EndoCase
 constR : EndoCase
 ident : EndoCase
 swap : EndoCase

interpret : EndoCase → Two → Two
interpret constL _ = L
interpret constR _ = R
interpret ident x = x
interpret swap L = R
interpret swap R = L

Pointwise : (Two → Two) → (Two → Two) → Set
Pointwise f g = (x : Two) → f x ≡ g x
```

The classifier simply inspects the two values of a function.

```
classify : (Two → Two) → EndoCase
classify f with f L | f R
... | L | L = constL
... | R | R = constR
... | L | R = ident
... | R | L = swap

classify-sound : (f : Two → Two) → Pointwise f (interpret (classify f))
classify-sound f L with f L | f R
... | L | L = refl
... | R | R = refl
... | L | R = refl
... | R | L = refl
classify-sound f R with f L | f R
... | L | L = refl
... | R | R = refl
... | L | R = refl
... | R | L = refl
```

Uniqueness follows by evaluating at L and R. Any attempted identification of two different cases forces either L to equal R, or conversely.

```
case-unique : (c d : EndoCase) → Pointwise (interpret c) (interpret d) → c ≡ d
case-unique constL constL _ = refl
case-unique constL constR p = ⊥-elim (L≠R (p L))
```

```
case-unique constL ident  p = ⊥-elim (L≠R (p R))
case-unique constL swap   p = ⊥-elim (L≠R (p L))
case-unique constR constL p = ⊥-elim (R≠L (p L))
case-unique constR constR _ = refl
case-unique constR ident  p = ⊥-elim (R≠L (p L))
case-unique constR swap   p = ⊥-elim (R≠L (p R))
case-unique ident  constL p = ⊥-elim (R≠L (p R))
case-unique ident  constR p = ⊥-elim (L≠R (p L))
case-unique ident  ident  _ = refl
case-unique ident  swap   p = ⊥-elim (L≠R (p L))
case-unique swap   constL p = ⊥-elim (R≠L (p L))
case-unique swap   constR p = ⊥-elim (L≠R (p R))
case-unique swap   ident  p = ⊥-elim (R≠L (p L))
case-unique swap   swap   _ = refl

classify-unique :
 (f : Two → Two) → (c : EndoCase) → Pointwise f (interpret c) → classify f ≡ c
classify-unique f c p =
 case-unique (classify f) c
  (λ x → trans (sym (classify-sound f x)) (p x))
```

6 Why the Closure is K_4

Composition of endomorphisms is total: for any two endomorphism cases, their composite is again an endomorphism of Two, hence is one of the four cases just classified.

```
_∘₂_ : (Two → Two) → (Two → Two) → Two → Two
(f ∘₂ g) x = f (g x)

composeCase : EndoCase → EndoCase → EndoCase
composeCase c d = classify (interpret c ∘₂ interpret d)

endomorphisms-closed :
 (c d : EndoCase) → Σ EndoCase (λ e → Pointwise (interpret e) (interpret c ∘₂ interpret d))
endomorphisms-closed c d =
 composeCase c d , λ x → sym (classify-sound (interpret c ∘₂ interpret d) x)
```

The closure argument is then finite and rigid. To represent the four endomorphism cases as distinct and usable data, no additional structure is assumed beyond what representation itself requires. Unpacking this requirement yields exactly the following three conditions.

1. *Separation.* Distinct endomorphism cases must remain distinct on the carrier. No identification of cases is allowed.
2. *Realisation.* Every endomorphism case must be represented by a vertex of the carrier.
3. *No surplus.* Every vertex of the carrier must arise from one of these cases.

These conditions are the representation contract used in this note. They make explicit what "representation" means here: each endomorphism case is carried as a distinct and retrievable element of the carrier, and every element of the carrier corresponds to such a case. Without separation, cases collapse; without realisation, cases are missing; without the no-surplus condition, additional structure is introduced. The contract is argued and displayed; it is not disguised as a theorem that precedes its own definition.

Under these three conditions, fewer than four vertices cannot represent the four cases without identification. If two cases are identified, case-unique shows that the corresponding functions agree pointwise; but the only way different rows of the four case table agree is by forcing L and R to coincide, contradicting L≠R. Thus separation and realisation together force at least four vertices.

An additional vertex is not forced by the endomorphism data. The composite of any two represented cases is already one of the four classified cases, by endomorphisms-closed. A new vertex would not be the value of any forced composition; it would violate no surplus.

Finally, total composability is not another assumption. Any ordered pair of endomorphisms has a composite, so there is no missing composability relation between distinct represented cases. For the underlying simple graph, this is exactly K_4.

The following small records package the three conditions. The record FaithfulClosure contains realisation and separation: realize supplies a vertex for each case, and reflect-realize prevents two cases from being identified. The record MinimalClosure adds no surplus through no-extra.

These conditions are therefore not independent choices once this notion of representation has been fixed; they unpack that notion.

```
record FaithfulClosure : Set₁ where
 field
  V       : Set
  realize : EndoCase → V
  reflect : V → EndoCase
  reflect-realize : (c : EndoCase) → reflect (realize c) ≡ c

record MinimalClosure : Set₁ where
 field
  closure : FaithfulClosure
  no-extra : (v : FaithfulClosure.V closure) →
             Σ EndoCase (λ c → FaithfulClosure.realize closure c ≡ v)

K4Vertex : Set
K4Vertex = EndoCase

K4Edge : K4Vertex → K4Vertex → Set
K4Edge c d = ¬ (c ≡ d)

canonicalK4 : MinimalClosure
canonicalK4 = record
 { closure = record
    { V       = EndoCase
    ; realize = λ c → c
    ; reflect = λ c → c
    ; reflect-realize = λ c → refl
    }
 ; no-extra = λ v → v , refl
 }

realize-reflect :
 (m : MinimalClosure) →
 (v : FaithfulClosure.V (MinimalClosure.closure m)) →
 FaithfulClosure.realize (MinimalClosure.closure m)
  (FaithfulClosure.reflect (MinimalClosure.closure m) v)
 ≡ v
realize-reflect m v with MinimalClosure.no-extra m v
... | c , p = trans (sym (cong realize q)) p
 where
  fc = MinimalClosure.closure m
  realize = FaithfulClosure.realize fc
  reflect = FaithfulClosure.reflect fc
  reflect-realize = FaithfulClosure.reflect-realize fc

  q : c ≡ reflect v
  q = trans (sym (reflect-realize c)) (cong reflect p)
```

The same information can be displayed as a compact K_4 presentation. This is not the large invariant record of the main kernel; it is the companion-level graph layer forced by any minimal representation of the four endomorphism cases. Vertices are the carrier of the closure, edges connect vertices whose reflected cases are distinct, and the two roundtrips certify that no case is lost and no vertex is surplus.

```
record K4Presentation : Set₁ where
 field
  vertices              : Set
  edge                  : vertices → vertices → Set
  realize-vertex        : EndoCase → vertices
  reflect-vertex        : vertices → EndoCase
  reflect-realize-vertex : (c : EndoCase) → reflect-vertex (realize-vertex c) ≡ c
  realize-reflect-vertex : (v : vertices) → realize-vertex (reflect-vertex v) ≡ v
  edge-complete         : (v w : vertices) → edge v w ≡ (¬ (reflect-vertex v ≡ reflect-vertex w))
  represented-closure : MinimalClosure

minimal-closure-presentation : (m : MinimalClosure) → K4Presentation
minimal-closure-presentation m = record
 { vertices              = FaithfulClosure.V fc
 ; edge                  = λ v w → ¬ (FaithfulClosure.reflect fc v ≡ FaithfulClosure.reflect fc w)
 ; realize-vertex        = FaithfulClosure.realize fc
 ; reflect-vertex        = FaithfulClosure.reflect fc
 ; reflect-realize-vertex = FaithfulClosure.reflect-realize fc
 ; realize-reflect-vertex = realize-reflect m
 ; edge-complete         = λ _ _ → refl
 ; represented-closure = m
 }
 where
  fc = MinimalClosure.closure m

canonicalK4Presentation : K4Presentation
canonicalK4Presentation = minimal-closure-presentation canonicalK4
```

The three record fields are not three independent stipulations. Each one is exactly the load-bearing piece of one of the three representation conditions, and the following lemmas name that load. The proofs are short because the conditions are exactly the bijection data; that is the point.

Separation is load-bearing. The field reflect-realize forces realize to be injective. If two cases were identified on the carrier, applying reflect to the identification would identify the cases themselves, contradicting the four-case enumeration of EndoCase.

```
realize-injective :
 (m : MinimalClosure) →
 (c d : EndoCase) →
 FaithfulClosure.realize (MinimalClosure.closure m) c
  ≡ FaithfulClosure.realize (MinimalClosure.closure m) d →
 c ≡ d
realize-injective m c d p =
 trans (sym (FaithfulClosure.reflect-realize (MinimalClosure.closure m) c))
  (trans (cong (FaithfulClosure.reflect (MinimalClosure.closure m)) p)
   (FaithfulClosure.reflect-realize (MinimalClosure.closure m) d))
```

Realisation is load-bearing. The totality of realize on EndoCase is exactly the demand that every endomorphism case is hosted by some vertex. Without it, some case has no representative and the carrier fails to support the composition table at all.

```
case-has-vertex :
 (m : MinimalClosure) →
 (c : EndoCase) →
 Σ (FaithfulClosure.V (MinimalClosure.closure m))
    (λ v → FaithfulClosure.realize (MinimalClosure.closure m) c ≡ v)
case-has-vertex m c =
 FaithfulClosure.realize (MinimalClosure.closure m) c , refl
```

No surplus is load-bearing. The field no-extra forbids vertices that do not arise from any endomorphism case. Without it, the carrier may host data that is not forced by the endomorphism set, breaking the claim that the closure adds nothing to its datum.

```
vertex-from-case :
 (m : MinimalClosure) →
 (v : FaithfulClosure.V (MinimalClosure.closure m)) →
```

```
Σ EndoCase
  (λ c → FaithfulClosure.realize (MinimalClosure.closure m) c ≡ v)
vertex-from-case m v = MinimalClosure.no-extra m v
```

Dropping reflect-realize re-admits identification of cases. Dropping the totality of realize re-admits cases without vertices. Dropping no-extra re-admits vertices without cases. In each case, what survives is no longer a representation of the four-case set in the intended sense. The three fields are therefore not chosen because they yield K_4; within this contract they are fixed because dropping any one breaks representation, and K_4 follows.

The companion proves these three load-bearing lemmas against its own slim records FaithfulClosure and MinimalClosure so that the file remains self-contained and type-checks without the larger source. The full development `Void.lagda.tex` carries the analogous statements against the richer K4Record used there; the present file does not import them, it reproduces them at the level of detail that this note needs.

This is the sense in which the closure yields K_4: the forced vertex set is the four-case set, and the induced simple graph connects every distinct pair. The equations reflect-realize and realize-reflect show that any carrier satisfying the three conditions is equivalent to the four-case carrier.

Equivalently, for any m : MinimalClosure, the carrier

$$\text{FaithfulClosure.V}\,(\text{MinimalClosure.closure}\,m)$$

is in bijection with EndoCase via realize and reflect. The field reflect-realize proves one composite to be the identity on EndoCase; realize-reflect proves the other composite to be the identity on the carrier. If composition on the carrier is read through this bijection,

$$v \star_m w = \text{realize}_m(\text{composeCase}\,(\text{reflect}_m\,v)(\text{reflect}_m\,w)),$$

then reflecting the composite recovers the four-case composition table of EndoCase. In this precise sense every minimal closure satisfying separation, realisation, and no surplus is isomorphic to the canonical four-case carrier. Thus K_4 is not only minimal; it is unique up to isomorphism under the stated constraints.

7 Leibniz collapse, executed

The earlier section on why two cannot occur without separation argued that the principle is universal: it appears in classical logic as substitutivity, in set theory as extensionality, in category theory as Yoneda, in quantum mechanics as the indistinguishability that forces the statistical structure. Once a concrete carrier is chosen, the same principle becomes a checkable term. Martin-Löf type theory realises it in two lines: the induction principle for propositional equality yields, by direct application, the Leibniz collapse. If two elements satisfy exactly the same predicates, they are propositionally equal.

```
leibniz-collapse :
{S : Set} (a b : S) →
((P : S → Set) → P a → P b) →
a ≡ b
leibniz-collapse a b ind = ind (λ x → a ≡ x) refl
```

The Agda term is one realisation of the principle, not its source. The principle is older than the type checker and would survive any particular choice of carrier; the type checker only confirms that the realisation in this carrier introduces no hidden step. Inside the present setting, this means that no element of any Distinction is silently a third position pretending to be a fourth: the separated field already certifies that the two boundary points cannot be Leibniz-collapsed onto each other, and the cover field certifies that no further occurrence escapes the two.

8 Re-projection to the datum

The K_4 closure does not replace the original distinction; it presupposes it. This section makes the dependency explicit at two levels of strength so that no reader can mistake the closure for an independent foundation, and so that the trivial direction is not advertised as more than it is.

The structural content as a checked term. The bijection between the carrier of any minimal closure and the four endomorphism cases is not left to commentary. It is bundled as a record and constructed from the lemmas already proved: reflect-realize is one round-trip, realize-reflect is the other. This is the load-bearing structural map.

```
record Bijection (A B : Set) : Set where
 field
  to      : A → B
  from    : B → A
  to-from : (b : B) → to (from b) ≡ b
  from-to : (a : A) → from (to a) ≡ a

closure-bijection-EndoCase :
 (m : MinimalClosure) →
 Bijection (FaithfulClosure.V (MinimalClosure.closure m)) EndoCase
closure-bijection-EndoCase m = record
 { to     = FaithfulClosure.reflect  (MinimalClosure.closure m)
 ; from   = FaithfulClosure.realize (MinimalClosure.closure m)
 ; to-from = FaithfulClosure.reflect-realize (MinimalClosure.closure m)
 ; from-to = realize-reflect m
 }
```

This term is the actual dependency. Every minimal closure has a carrier in checked bijection with EndoCase; by two-normal-form, EndoCase is built from Two, the canonical carrier of any binary distinction. The four-vertex structure is therefore not an independent foundation; it is, up to the bijection just constructed, the endomorphism set of the normal form of any binary distinction.

Canonical re-projection. At the level of returning a value of type Distinction, every minimal closure re-projects to the canonical binary distinction. This is not a carrier-level isomorphism between four vertices and two points; such an isomorphism would be false. The carrier-level bijection is the checked map to EndoCase above. The re-projection below records the dependency at the Distinction level: the four-case closure is readable only through the binary normal form whose endomorphisms it represents.

```
k4-presupposes-distinction : MinimalClosure → Distinction
k4-presupposes-distinction _ = Two-distinction

record FormalClosureChain (d : Distinction) : Set₁ where
 field
  binary-normal-form        : DistinctionIso d Two-distinction
  equality-discipline       : NormalFormEqualityDiscipline d
  endomorphism-classifier   : (Two → Two) → EndoCase
  classifier-is-sound       : (f : Two → Two) →
   Pointwise f (interpret (endomorphism-classifier f))
  classifier-is-unique      :
   (f : Two → Two) (c : EndoCase) →
   Pointwise f (interpret c) → endomorphism-classifier f ≡ c
  composition-is-closed     :
   (c e : EndoCase) →
   Σ EndoCase (λ r → Pointwise (interpret r) (interpret c ∘₂ interpret e))
  k4-presentation           : K4Presentation
  closure-carrier-bijection : Bijection (K4Presentation.vertices k4-presentation) EndoCase
  reprojected-distinction   : Distinction
  reprojection-is-canonical : reprojected-distinction ≡ Two-distinction

theorem-formal-closure-chain :
 (d : Distinction) → FormalClosureChain d
theorem-formal-closure-chain d = record
```

```
{ binary-normal-form       = two-normal-form d
; equality-discipline      = theorem-normal-form-equality-discipline d
; endomorphism-classifier  = classify
; classifier-is-sound      = classify-sound
; classifier-is-unique     = classify-unique
; composition-is-closed    = endomorphisms-closed
; k4-presentation          = canonicalK4Presentation
; closure-carrier-bijection = closure-bijection-EndoCase canonicalK4
; reprojected-distinction  = k4-presupposes-distinction canonicalK4
; reprojection-is-canonical = refl
}

record Dn2FormalClosureChain (d : Distinction-n 2) : Set₁ where
 field
  n2-equivalence : DistinctionDn2Equivalence
  left-is-point-0  : Distinction.left (Dn2-to-distinction d) ≡ Distinction-n.point d fz
  right-is-point-1 : Distinction.right (Dn2-to-distinction d) ≡ Distinction-n.point d (fs fz)
  closure-chain  : FormalClosureChain (Dn2-to-distinction d)

theorem-Dn2-formal-closure-chain :
 (d : Distinction-n 2) → Dn2FormalClosureChain d
theorem-Dn2-formal-closure-chain d = record
 { n2-equivalence = theorem-distinction-Dn2-equivalence
 ; left-is-point-0  = refl
 ; right-is-point-1 = refl
 ; closure-chain  = theorem-formal-closure-chain (Dn2-to-distinction d)
 }
```

The full formalisation uses a richer K4Record. The structural dependency carries over without change: the closure carrier is not an independent foundation, it is, by closure-bijection-EndoCase, in checked bijection with the endomorphism set of the binary normal form. The record theorem-formal-closure-chain bundles the route as one object, and theorem-Dn2-formal-closure-chain states the same route from the $n = 2$ template.

Part III. Frame

9 Scope Clarification

The scope of this note is narrow by design.

What is not claimed. Nothing in the preceding sections asserts that a binary distinction takes place as an event in nature. The development never builds an inhabitant of Distinction without supplied data; every Distinction-valued construction below the abstract takes a Distinction as input. The proofs are silent on the question of whether such an input exists in any external sense.

What is conceptually argued, not formally proved. The introductory paragraphs argue that any formalisation at all already presupposes a field of separability: relations, morphisms, judgements, and equations all require distinguishable positions before they can be used. This is a remark on the conditions under which formalisation is possible, not a theorem. Nothing in the Agda code attempts to prove it, and the formal results below do not depend on it: they begin once the data are granted.

What is machine-proved. The conditional consequence is what the Agda code establishes. Given a structure with two distinguishable, exhaustive points, the normal form is Two; given the endomorphisms of that normal form, representing the four cases with separation, realisation, and no surplus yields the four-case carrier with complete simple adjacency; and the closure entails the data from which it was built. The result therefore does not say that every theory begins with such a distinction, nor that every mathematical object reduces to Two. It says only what follows when the stated data are granted.

No further constructions are treated here. They are outside the present claim.

What the theory of formulability adds. The theory of formulability is the architectural thesis that this conditional formal chain is not an isolated trick, but the first stable skeleton of any theory capable of formulation. That thesis is not a hidden Agda lemma. It is the reading that makes the chain matter: the pre-formal boundary explains why the datum is unavoidable, the checked kernel explains what the datum forces, and *Form* tests how far the resulting invariant ledger can be read as a structure of physical description.

10 Placement

This section locates the contribution in three lineages so that the formal result is read at the right level.

Spencer-Brown. The thesis that distinction is the foundational act is at least as old as *Laws of Form* (1969). The present note neither defends nor extends that thesis. It records what holds once a distinction is granted, in a form a type checker can verify. The contribution is the precision of the conditional, not the originality of the antecedent.

Lawvere. "Adjointness in Foundations" (1969) treats truth and falsity as adjoint to the act of distinguishing. The present note operates one level below: before truth-values are assigned, before propositions are evaluated. Two is a fresh carrier, deliberately unread as $\{\bot, \top\}$, so that the closure result depends on separation alone and not on a prior reading of the two points as truth-values.

Univalent foundations. Two is an h-set, and the development uses `--without-K`. No higher-dimensional structure is assumed or invoked. The note is therefore compatible with both intensional Martin-Löf type theory and a univalent reading; nothing in the closure result depends on the choice of identity discipline.

What is offered here is not a competing foundation, and not an argument that distinction-based foundations are superior to others. It is one machine-checked statement at a place where earlier work has been mostly informal: that the closure of binary distinction is K_4, unique up to isomorphism, under three precisely fixed conditions of representation, and that this closure points back to the data from which it was built.

11 Outlook

The core result stops at the framework-forced closure of binary distinction. Its role in the larger development is not to prove everything at once, but to fix the first carrier on which the larger claim can be tested without handwaving. Once the four-case closure is fixed, further claims can be separated cleanly: some are formal consequences of the core, some are additional constructions over it, and some belong only to the interpretive layer.

The next formal step in *Void and Form* is therefore not a jump to a global conclusion. It is the controlled question of what can be built on the surviving carrier without adding new generators. This note only isolates the starting point.

Interpretive continuation

The later volume *Form* belongs here, at the end of the companion, not at the beginning. It is not the kernel and it is not evidence for the kernel. The direction is one-way: the checked kernel exports an invariant ledger; *Form* reads that ledger structurally and, where the reading is explicit enough, physically. A failed physical reading would not refute the normal-form theorem or the four-case

closure. It would refute that interpretation. A successful reading would not turn the interpretation into an Agda theorem; it would support the larger claim by showing that the formal skeleton has empirical reach.

This does not make *Form* decorative. Its empirical surface is deliberately exposed. The physical identifications are tested by correction chains, sigma deviations, and failure conditions. In the current ledger, the post-correction comparisons reported in `Form.lagda.tex` are:

Observable	Reported comparison in *Form*
α^{-1}	second-order correction $137+11/306+295/(306\cdot 137^2)$, within 0.4σ of CODATA; the alternative interpolation degrees are reported as 2445σ and 4891σ failures.
m_p/m_e	correction chain $949\sigma \to 20\sigma \to 0.08\sigma$.
m_μ/m_e	correction chain $36\sigma \to 13\sigma \to 0.12\sigma$.
m_τ/m_μ	remains closed at about 0.1σ.
$\sin^2\theta_W$	weak-mixing chain $7.7\sigma \to 5.0\sigma \to 3.5\sigma \to 0.001\sigma$.
Ω_m	K_4 gives $\Omega_m \approx 0.3183$, within 0.5σ of Planck; the K_3 and K_5 alternatives are reported as 10.9σ and 11.8σ away.

The same separation governs the open tests. The B-meson lepton-universality anomaly is read through the muon bridge factor $16/69 \approx 0.232$ as a Wilson-coefficient target. The doubly charmed baryon section reads the observed Ξ_{cc} pair as occupation of a forced two-slot doublet ledger, not as a mass prediction. The frontier register names exactly two gravitational-wave polarizations, a neutrino seesaw at loop depth 5, Planck-scale curvature $R = 12$, no fourth generation, and no extra spacetime dimension. These are interpretive and empirical claims. They are where the larger project becomes vulnerable to physics.

12 Navigation into the book

The corresponding self-contained development is in the repository de-johannes/Void-and-Form. The main file is `Void.lagda.tex`. The route through the larger book is:

- *Void as boundary*: *What Void Does Not Name*, *Why Formal Systems Cannot Begin Themselves*, and *Representation Requires A Cut*.
- *The binary threshold*: *Why The First Cut Is Binary*, *The Formal Threshold*, and the record `Distinction`.
- *Normal form and classification*: *Elimination and Classification*, *Two Normal Form*, and the endomorphism classification of `Two -> Two`.
- *Four-vertex closure*: *Graph Presentation*, *The K4 Invariant Record*, *K4 Representation Theory*, and `K4Record-is-canonical`.
- *Return to the datum*: `record-presupposes-distinction`, the formal witness that the surviving closure entails the originating distinction.
- *Interpretive continuation*: the later volume *Form*, where origin, contact, spatial separation, temporal order, physical reading, contingency, and induction are treated as structural interpretations rather than new generators.